

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes

GOUASMI Yassine
MOKHTARY Moumen Aya

Université Paris Nanterre
Master 1 MIAGE
Optimisation Combinatoire
Enseignant : François Delbot

09/05/2026

Dépôt GitHub :

<https://github.com/YassineGOUASMI/graphbench-project>

Résultat final : 100/100 contre-exemples trouvés

Score final approximatif : 241.75

Table des matières

1	Introduction	3
2	Architecture du projet	3
2.1	Structure générale	3
2.2	Rôle des principaux fichiers	3
3	Génération et mutations des graphes	4
3.1	Génération initiale	4
3.2	Mutations locales	4
4	Calcul des invariants	4
4.1	Invariants utilisés	4
5	Recherche évolutionnaire	5
5.1	Principe général	5
5.2	Critère de réussite	5
6	Recherche baseline	5
6.1	Extrait du moteur baseline	5
6.2	Capture d'exécution	6
7	Amélioration inspirée de FunSearch	6
7.1	Fonction de score	6
7.2	Sélection mixte	7
7.3	Optimisations	7

8 Résultats expérimentaux	7
8.1 Résultat final	7
8.2 Résumé	8
8.3 Capture du résultat final	8
9 Limites	8
10 Discussion scientifique	8
11 Conclusion	9

1 Introduction

Le challenge GraphBench consiste à rechercher automatiquement des contre-exemples à des conjectures en théorie des graphes.

L'objectif du projet est de construire un programme capable de générer automatiquement des graphes, de calculer leurs invariants puis de rechercher des graphes violant une conjecture donnée.

Pour cela, nous avons développé un moteur évolutionnaire basé sur :

- la génération aléatoire de graphes,
- des mutations locales,
- le calcul d'invariants,
- une recherche guidée par score heuristique,
- une sélection des meilleurs candidats.

Le système obtenu permet finalement de trouver automatiquement des contre-exemples pour les 100 premières conjectures testées du benchmark.

2 Architecture du projet

Le projet est organisé en plusieurs modules spécialisés.

2.1 Structure générale

```
graphbench-project/  
|-- benchmark/  
| |-- benchmark.xlsx  
|-- experiments/  
| |-- run_basic_search.py  
| |-- run_baseline_search.py  
| |-- run_final.py  
|-- results/  
| |-- basic_search/  
| |-- baseline_search/  
| |-- final/  
|-- src/  
| |-- conjecture.py  
| |-- generator.py  
| |-- invariants.py  
| |-- mutations.py  
| |-- search.py  
| |-- search_baseline.py  
| |-- scoring.py  
|-- requirements.txt  
|-- README.md
```

2.2 Rôle des principaux fichiers

- `generator.py` : génération initiale des graphes,
- `mutations.py` : opérateurs de mutation,
- `invariants.py` : calcul des invariants,
- `search.py` : moteur de recherche principal,
- `search_baseline.py` : version baseline,
- `scoring.py` : fonction de score heuristique,
- `run_basic_search.py` : expérience principale,

- `run_baseline_search.py` : expérience baseline,
- `run_final.py` : génération des résultats finaux.

3 Génération et mutations des graphes

3.1 Génération initiale

Le système génère des graphes connexes aléatoires servant de population initiale.

Plusieurs familles de graphes sont utilisées :

- graphes aléatoires,
- chemins,
- cycles,
- étoiles,
- graphes bipartis,
- cliques,
- graphes plus denses.

L'objectif est de démarrer la recherche avec des structures variées afin d'explorer plusieurs régions de l'espace des graphes.

3.2 Mutations locales

Les opérateurs de mutation implémentés sont :

- ajout d'arête,
- suppression d'arête,
- ajout de sommet,
- suppression de sommet,
- ajout de feuille,
- subdivision d'arête,
- ajout de chemin,
- ajout de clique,
- ajout de faux jumeau,
- densification locale.

Certaines itérations appliquent plusieurs mutations successives afin de sortir plus facilement des minima locaux.

4 Calcul des invariants

Le moteur calcule plusieurs invariants de graphes à l'aide de NetworkX.

4.1 Invariants utilisés

- nombre de sommets,
- nombre d'arêtes,
- degré minimum,
- degré maximum,
- degré moyen,
- densité,
- diamètre,
- rayon,
- proximity,
- remoteness,
- nombre de triangles,

- clique maximum,
- nombre d'indépendance,
- couverture par sommets,
- couplage maximum,
- domination,
- domination indépendante,
- domination totale,
- connectivité par sommets,
- connectivité par arêtes.

Afin d'accélérer les expériences, le programme ne calcule que les invariants réellement nécessaires pour la conjecture courante.

5 Recherche évolutionnaire

5.1 Principe général

Le système maintient une population de graphes candidats.

À chaque itération :

1. un graphe parent est sélectionné,
2. une ou plusieurs mutations sont appliquées,
3. les invariants sont recalculés,
4. la violation de la conjecture est évaluée.

Les meilleurs graphes sont ensuite conservés dans la population.

Aucun contre-exemple connu n'a été codé manuellement dans le programme. Les graphes sont entièrement reconstruits automatiquement par la recherche.

5.2 Critère de réussite

Une conjecture est réfutée lorsque :

$$\text{violation}(G) > 0$$

Le graphe correspondant devient alors un contre-exemple valide.

6 Recherche baseline

La baseline utilise directement la violation réelle comme score principal.

6.1 Extrait du moteur baseline

```
score = conjecture.violation(invariants)

if score > best_score:
    best_score = score
```

Cette version fonctionne correctement sur les conjectures simples mais devient plus instable sur les conjectures difficiles.

6.2 Capture d'exécution

```
--- Restart 1/3 ---
Score initial : -0.75
Nouveau meilleur score : -0.5
Nouveau meilleur score : -0.25
Nouveau meilleur score : 0.0
Nouveau meilleur score : 0.25

===== MEILLEUR RÉSULTAT BASELINE =====
Trouvé : True
Score : 0.25
Temps : 2.04 sec
Coût : 2.04

=====
Conjecture 2117
If G is a connected graph,  $x = \text{matching\_number}$  and  $y = \text{independent\_}$ 
 $7 * x(G)^2 + 1/3 * x(G)^3$ 
=====

--- Restart 1/3 ---
Score initial : -3.2275714285714194
Nouveau meilleur score : -2.2275714285714194
Nouveau meilleur score : -1.2275714285714194
Nouveau meilleur score : -0.5132857142857077
Nouveau meilleur score : 0.4867142857142923

===== MEILLEUR RÉSULTAT BASELINE =====
Trouvé : True
Score : 0.4867142857142923
Temps : 0.54 sec
Coût : 0.54
```

FIGURE 1 – Exécution de la recherche baseline

7 Amélioration inspirée de FunSearch

Après la baseline, nous avons ajouté une recherche guidée par une fonction de score heuristique inspirée de FunSearch.

L'idée est de ne plus sélectionner les graphes uniquement selon la violation réelle, mais aussi selon plusieurs caractéristiques structurelles du graphe.

7.1 Fonction de score

La fonction de score finale dépend :

- de la violation réelle,
- du signe de la conjecture,
- des variables x et y ,
- de plusieurs invariants du graphe.

```

score = 100000.0 * violation

if conjecture.sign == "<=":
    score += 1.0 * y
    score -= 0.2 * x

score += 0.25 * clique
score += 0.12 * matching
score += 0.12 * vertex_cover
score -= 0.005 * n

```

La violation réelle reste largement prioritaire, tandis que les autres termes servent surtout à guider l'exploration.

7.2 Sélection mixte

Le moteur conserve deux types de graphes :

- les graphes ayant la meilleure violation réelle,
- les graphes ayant le meilleur score heuristique.

```

top_real = sorted(population,
                  key=lambda x: x[0],
                  reverse=True)

top_heuristic = sorted(population,
                      key=lambda x: x[1],
                      reverse=True)

```

Cette sélection mixte permet de garder de la diversité dans la population tout en privilégiant les vrais contre-exemples.

7.3 Optimisations

Plusieurs optimisations ont été ajoutées :

- cache des invariants déjà calculés,
- calcul partiel des invariants,
- mutations multiples,
- diversification de la population,
- génération de graphes spécialisés selon les invariants de la conjecture.

Ces optimisations ont permis d'améliorer fortement les performances sur les conjectures difficiles.

8 Résultats expérimentaux

8.1 Résultat final

Les expériences ont été réalisées sur les 100 premières conjectures du benchmark.

Le système a finalement obtenu :

100 / 100 contre-exemples trouvés

8.2 Résumé

Mesure	Valeur
Conjectures testées	100
Contre-exemples trouvés	100
Taux de réussite	100%
Coût approximatif	241.75
Temps moyen	2.42 sec

TABLE 1 – Résumé final des performances

8.3 Capture du résultat final

```
Coût : 0.08

===== RÉSUMÉ FINAL =====
Conjectures testées : 100
Contre-exemples trouvés : 100 / 100
Taux de réussite : 100.0 %
Score total approximatif : 241.75
Temps moyen : 2.42 sec

Résultats sauvegardés dans results/basic_search/
Résumé sauvegardé dans results/basic_search/summary.txt
```

FIGURE 2 – Résultat final obtenu

9 Limites

Malgré les bons résultats obtenus, plusieurs limites subsistent :

- forte dépendance à l'aléatoire,
- coût croissant pour certaines conjectures,
- absence de génération automatique de fonctions heuristiques,
- absence d'apprentissage automatique réel.

10 Discussion scientifique

Les conjectures les plus faciles à réfuter concernent souvent des invariants locaux comme le degré maximum, les triangles ou les cliques.

À l'inverse, les invariants globaux comme la domination totale, la proximité ou certaines mesures spectrales sont plus difficiles à optimiser car ils dépendent fortement de la structure globale du graphe.

Les mutations les plus efficaces ont été :

- l'ajout de clique,
- la densification locale,
- l'ajout de faux jumeaux,
- les mutations multiples.

L'approche inspirée de FunSearch a clairement amélioré les performances par rapport à la baseline classique. La sélection mixte et le score heuristique ont permis d'obtenir les 100 contre-exemples sans utiliser de ciblage manuel des conjectures.

L'utilisation de l'IA générative a principalement servi à proposer des idées d'heuristiques, améliorer l'organisation du projet et accélérer certaines phases de développement. Les meilleurs résultats ont ensuite été obtenus après plusieurs essais expérimentaux et ajustements manuels.

11 Conclusion

Ce projet a permis de construire un moteur complet de recherche automatique de contre-exemples en théorie des graphes.

Les principaux apports du projet sont :

- une architecture modulaire,
- un moteur évolutionnaire robuste,
- une recherche guidée par score heuristique,
- plusieurs optimisations de performance,
- une approche inspirée de FunSearch.

Les résultats obtenus montrent qu'une approche évolutionnaire relativement simple peut produire de très bonnes performances sur un benchmark de conjectures.

Des améliorations futures pourraient inclure :

- génération automatique de nouvelles fonctions de score,
- apprentissage automatique,
- mutations guidées dynamiquement,
- modèles génératifs de graphes.